SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1108

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Ramamoorthy

QUESTIONS: 6 **Total Marks:30**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Deepa lakshana.B (192471054) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1.Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

Class

A class is a blueprint or template used to create objects. It defines the data members (attributes) and member functions (methods).
Example: BankAccount class contains account number, balance, deposit(), withdraw().

Object

An object is an instance of a class that contains actual values and performs actions.
Example: Customer account "A12345" is an object of the BankAccount class.

Message Passing

Message passing is the communication between objects through method calls.
Example: A Customer object sends a message to the BankAccount object to transfer money.

Encapsulation

Encapsulation combines data and methods into a single unit and protects data using access modifiers.
Example: Balance is private and updated only through deposit() and withdraw() methods.

Polymorphism

Polymorphism allows one interface to perform different tasks.
Example: calculateInterest() works differently for Savings Account and Current Account.

(b) Describe the following object relationships:

Association

A general relationship where two objects communicate.
Example: Customer uses Bank Account.

Aggregation

A weak "has-a" relationship where objects can exist independently.
Example: Bank has many Customers.

Composition

A strong "part-of" relationship where the child object cannot exist without the parent.
Example: Transaction History belongs to a Bank Account.

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

- Promotes code reusability.
- Improves maintainability.
- Enhances security using encapsulation.
- Simplifies future modifications.
- Makes the banking system scalable and flexible.

2.Consider developing a **Smart Healthcare Management System**.

(a) Explain the following object-oriented design principles:

Abstraction

Shows only essential details while hiding implementation.
Example: Doctor views patient records without knowing database implementation.

Encapsulation

Protects patient information by restricting direct access.

Modularity

Divides the system into modules such as Patient, Doctor, Billing, and Pharmacy.

(b) Describe the following SOLID principles:

Liskov Substitution Principle (LSP)

Derived classes should replace base classes without affecting program correctness.

Example: SpecialistDoctor can replace Doctor without changing system behavior.

Interface Segregation Principle (ISP)

Clients should not be forced to implement unnecessary methods.

Example: Doctor interface and Receptionist interface are separate.

(c) Explain how these principles improve software quality and flexibility.

- Better flexibility
- Easier maintenance
- Improved code quality
- Increased scalability
- Reduced coupling

3.Suppose a software company is developing an **Online Food Delivery System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

Planning

Defines project goals, scope, budget, and feasibility.

Requirement Analysis

Collects customer requirements such as ordering, payment, and delivery.

Design

Creates system architecture, database, and UML diagrams.

Development

Developers write the program code.

Testing

Finds and fixes software defects.

Maintenance

Updates software and fixes issues after deployment.

(b) Describe the purpose of each SDLC phase.

- Planning → Project roadmap
- Requirement Analysis → Understand customer needs
- Design → Create system structure
- Development → Build software
- Testing → Ensure quality
- Maintenance → Improve and update software

(c) Explain how SDLC contributes to delivering reliable software.

- Produces reliable software

- Reduces project risks
- Improves quality
- Ensures customer satisfaction
- Enables timely delivery

4.Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC models:

Prototype Model

Develops a working prototype before the final product.

Spiral Model

Combines iterative development with risk analysis.

Agile Model

Develops software in small iterations with continuous customer feedback.

DevOps Model

Integrates development and operations for continuous integration and deployment.

(b) Compare the Prototype Model and Agile Model.

Prototype Model	**Agile Model**
Builds initial prototype	Builds working software in iterations
Used for unclear requirements	Used for changing requirements
Customer reviews prototype	Customer participates throughout
Less flexible after prototype	Highly flexible

(c) Recommend the most suitable SDLC model for this project with justification.

Agile Model

Justification:

- Requirements change frequently.
- Continuous customer feedback.
- Faster delivery.
- Easy modifications.
- Better quality through iterative testing.

5.Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

- Visualizes software design.
- Improves communication.
- Simplifies system understanding.
- Helps documentation.
- Supports object-oriented development.

(b) Describe the following UML diagrams:

State Diagram

Shows different states of an object.

Example: Vehicle → Available → Booked → In Transit → Delivered.

Component Diagram

Shows software components and their relationships.

Deployment Diagram

Shows hardware and software deployment.

Package Diagram

Groups related classes into packages.

(c) Explain how UML models improve communication among stakeholders.

- Better communication among developers and clients.
- Reduces misunderstandings.
- Helps project planning.
- Improves documentation.
- Makes maintenance easier.

6.Consider developing a **Smart Inventory Management System**.

(a) Explain the concept of modular software design.

Modular design divides software into independent modules that perform specific tasks.

Example: Inventory Module, Billing Module, Supplier Module.

(b) Compare the procedural programming approach with the object-oriented programming approach.

Procedural Programming	**Object-Oriented Programming**
Function-based	Object-based
Data is less secure	Data is protected using encapsulation
Difficult to reuse	High code reusability
Less maintainable	Easier maintenance
Top-down approach	Bottom-up approach

(c) Explain how modularity improves software scalability, maintainability, and reusability.

Scalability

New modules can be added without affecting existing modules.

Maintainability

Errors are easier to locate and fix.

Reusability

Modules can be reused in other applications.

Additional Benefits

- Easier testing
- Reduced development time
- Better collaboration among developers

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis

Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps